Understanding PWM

PWM-Part-II

Amer Iqbal Qureshi | Mikrotronics Pakistan

Understanding Pulse Width Modulation

How PWM Works

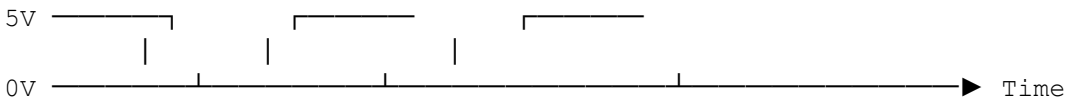
Turning Digital Pulses into Useful Power

Imagine watching a ceiling fan from a distance. If someone quickly switches its power ON and OFF every few milliseconds, the fan does not start and stop abruptly. Instead, because of its inertia, it continues rotating smoothly. If the power remains ON for a longer portion of each cycle, the fan spins faster. If the ON time is shortened, the fan gradually slows down.

This is precisely how Pulse Width Modulation works.

A PWM signal does not change the output voltage. Instead, it repeatedly switches the output between **HIGH** and **LOW** at a fixed frequency. By changing the amount of time the signal remains HIGH during each cycle, the microcontroller controls the average power delivered to the load.

This concept is illustrated below.



Although the output is constantly switching between 0 V and 5 V, many electronic devices respond to the **average energy** rather than to the individual pulses.

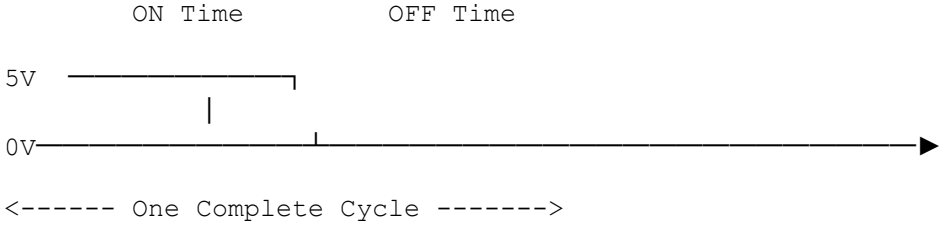
This simple principle makes PWM one of the most powerful techniques available to an embedded systems engineer.

The Anatomy of a PWM Signal

Every PWM waveform consists of two parts.

- **ON Time (TON)** — the duration for which the signal remains HIGH.
- **OFF Time (TOFF)** — the duration for which the signal remains LOW.

Together they form one complete PWM cycle.



The total duration of one complete cycle is called the **Period (T)**.

Mathematically,

```
Period = ON Time + OFF Time
```

Every PWM waveform repeats this cycle continuously.

Duty Cycle – The Heart of PWM

The most important characteristic of a PWM signal is its **Duty Cycle**. The duty cycle tells us what percentage of each cycle the signal remains HIGH.

It is calculated as:

```
Duty Cycle (%) = (ON Time / Period) × 100
```

If the signal remains HIGH for half of the period,

```
Duty Cycle = 50%
```

If it remains HIGH for only one-quarter of the period,

```
Duty Cycle = 25%
```

Similarly,

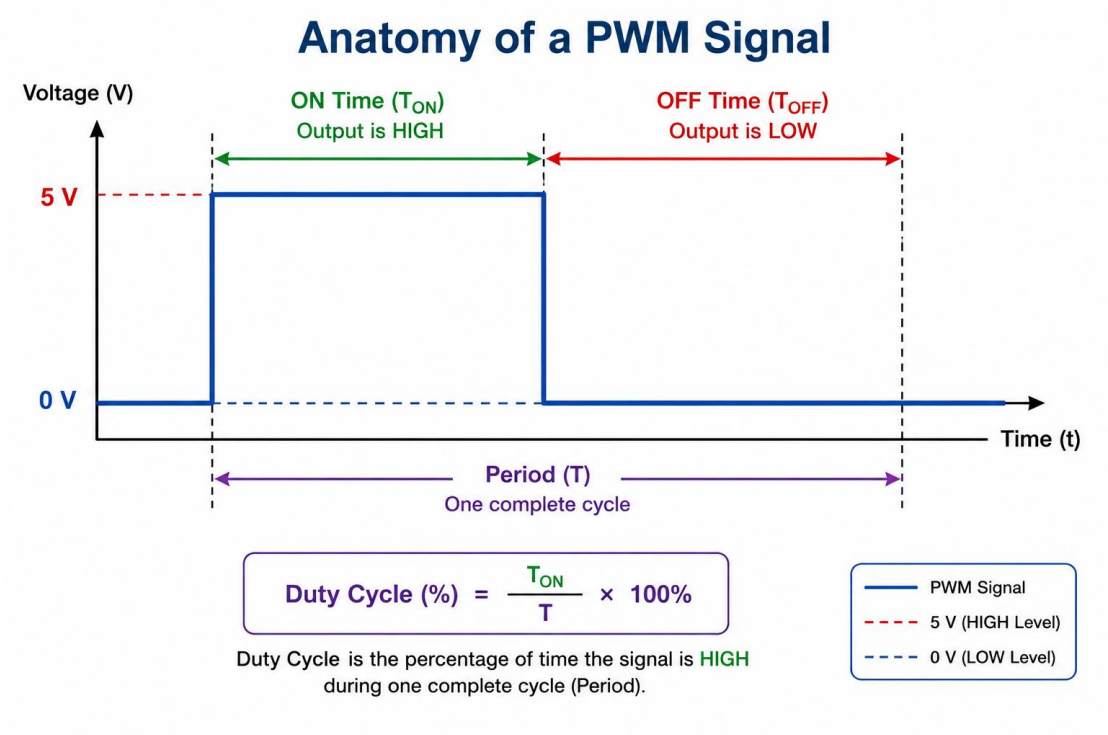
```
100% Duty Cycle
```

```
Output always HIGH
0% Duty Cycle
```



```
Output always LOW
```

The duty cycle therefore determines how much electrical energy is delivered to the load.



Understanding Duty Cycle with Examples

Let us consider a PWM signal operating from a 5 V supply.

20% Duty Cycle

████_____

The output remains HIGH only 20% of the time.

Average Voltage

```
5V × 20%
```

```
= 1.0V
```

An LED would appear dim. A motor would rotate slowly.

50% Duty Cycle

████████_____

Average Voltage

```
5V × 50%
```

```
= 2.5V
```

The LED glows at medium brightness. The motor rotates at approximately half speed.

80% Duty Cycle

Average Voltage

```
5V × 80%

= 4.0V
```

The LED appears much brighter. The motor runs close to full speed.

100% Duty Cycle

The output remains continuously HIGH. Average Voltage

```
5V
```

This is equivalent to connecting the load directly to the power supply.

Average Voltage Produced by PWM

Although the PWM signal rapidly alternates between 0 V and 5 V, the **average voltage** depends only on the duty cycle.

Duty Cycle	Average Voltage (5 V Supply)
0%	0.0 V
10%	0.5 V
20%	1.0 V
30%	1.5 V
40%	2.0 V
50%	2.5 V
60%	3.0 V
70%	3.5 V
80%	4.0 V
90%	4.5 V
100%	5.0 V

This relationship is linear and can be expressed as

```
Average Voltage = Supply Voltage × Duty Cycle
```

where the duty cycle is expressed as a fraction (for example, 0.5 for 50%).

It is important to remember that **this is an average value**. The microcontroller output is still switching between 0 V and 5 V; it is not generating a true DC voltage.

How Different Loads Respond to PWM

Not every electronic device reacts to PWM in the same way.

LEDs

An LED responds almost instantly to electrical current, but the human eye cannot detect extremely rapid flashing. Instead of seeing the LED blink, we perceive a continuous brightness proportional to the duty cycle.

DC Motors

Motors possess rotational inertia. The rotor cannot accelerate and decelerate during every PWM pulse. Instead, it averages the applied energy, producing smooth speed control.

Heating Elements

A heating element has significant thermal inertia. Its temperature changes much more slowly than the PWM switching frequency. As a result, the heat output corresponds closely to the average power delivered.

Audio Systems

PWM can also be used to generate audio signals. When passed through suitable filters or amplifier circuits, PWM can produce tones, speech, or even music. Later in this chapter, we shall generate sine waves and arbitrary waveforms using the MikroDuino PWM SDK.

Why PWM is So Efficient

Suppose we wish to reduce the brightness of an LED. One option is to place a resistor in series with the LED. The resistor converts unwanted electrical energy into heat.

PWM takes a completely different approach. The output transistor inside the microcontroller is either

- Fully ON, or
- Fully OFF.

In these two states, very little power is dissipated inside the switching device.

Consequently,

- Less energy is wasted.
- Less heat is generated.
- Battery life is improved.
- Power supplies become more efficient.

This is one of the principal reasons why PWM is used extensively in battery-powered devices, electric vehicles, robotics, and industrial control systems.

Frequency vs Duty Cycle

Two Parameters That Beginners Often Confuse

After learning about PWM, beginners frequently ask a question:

> **"If I want my LED to become brighter, should I increase the frequency?"**

The answer is **No**.

Brightness is primarily controlled by the **duty cycle**, not by the frequency. Similarly, increasing the duty cycle does not necessarily change the switching frequency.

These are two entirely different characteristics of a PWM signal. Understanding the difference is essential before designing any embedded system.

What is PWM Frequency?

The **frequency** of a PWM signal indicates how many complete cycles occur every second. It is measured in **Hertz (Hz)**.

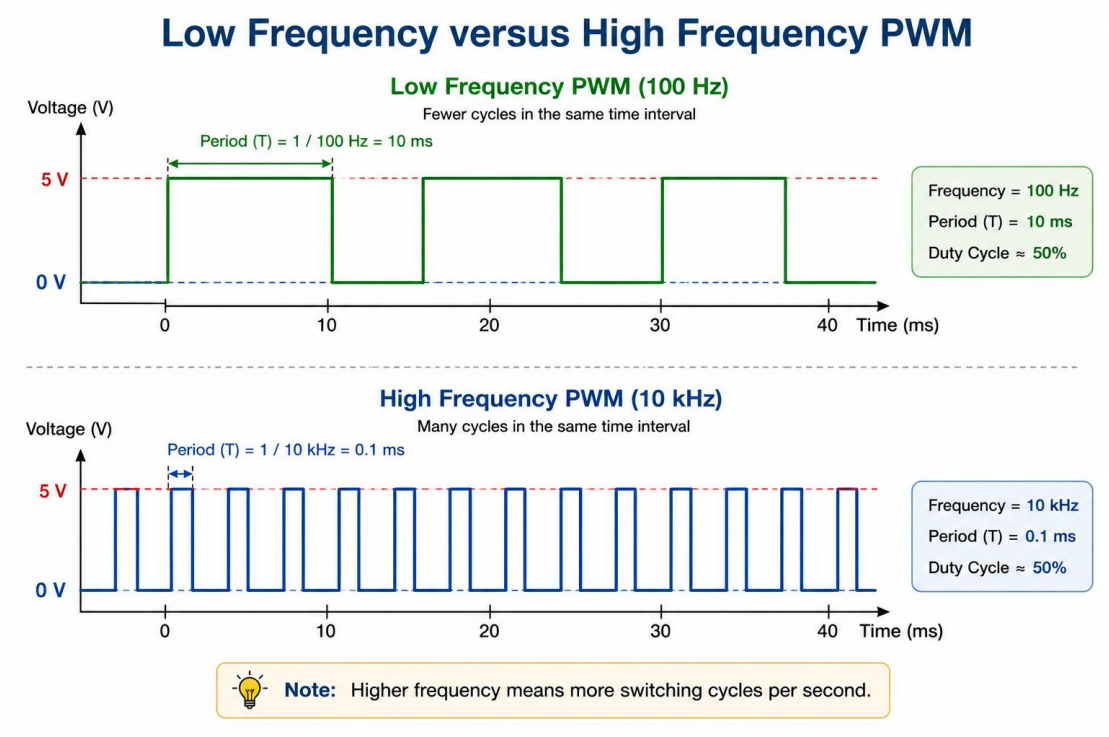
For example,

- 100 Hz means the waveform repeats 100 times every second.
- 1 kHz means 1,000 repetitions per second.
- 20 kHz means 20,000 repetitions every second.

Frequency and period are inversely related.

$f = 1/T1$

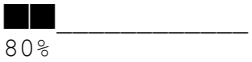
As the frequency increases, each PWM cycle becomes shorter.



Duty Cycle Controls Power

Consider two PWM signals operating at exactly the same frequency.

20%



80%

Although both signals switch at the same rate, the second waveform delivers much more energy because it remains HIGH for a larger portion of each cycle.

Therefore,

- Increasing duty cycle increases average power.
- Decreasing duty cycle reduces average power.

Frequency Controls Switching Behaviour

Now consider two signals with exactly the same duty cycle but different frequencies.

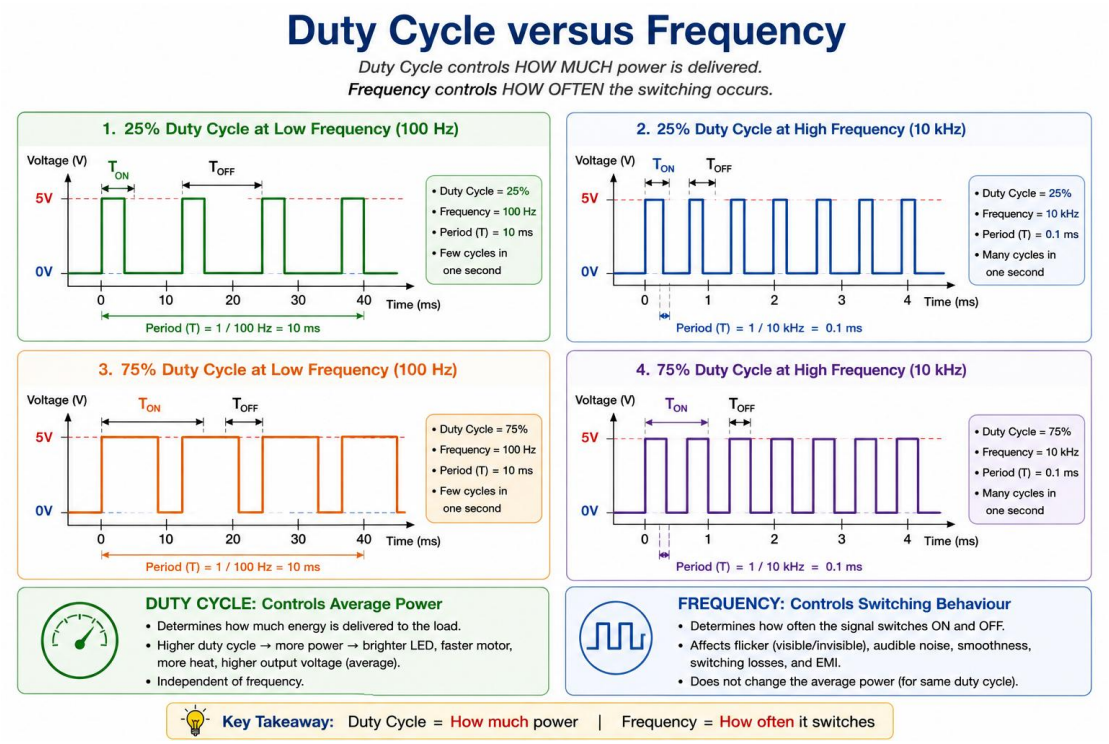
`50% at 100 Hz and 50% at 10 kHz`

Both signals deliver approximately the same average power. However, their behaviour is very different.

A low-frequency LED may visibly flicker. A high-frequency LED appears perfectly steady.

A motor operating at low frequency may vibrate and produce audible noise. At higher frequencies, the motor runs much more smoothly and quietly.

Thus, frequency affects **how** the energy is delivered, while duty cycle determines **how much** energy is delivered.



Practical Examples

The influence of duty cycle and frequency can be observed in many embedded applications.

LED Brightness

- Duty Cycle determines brightness.
- Frequency determines whether flicker is visible.

DC Motor

- Duty Cycle determines motor speed.
- Frequency affects smoothness and audible noise.

Servo Motor

For hobby servos, the frequency is generally fixed at **50 Hz**.

The servo position is determined by the **pulse width**, not by changing the frequency.

Switching Power Supplies

In switching regulators, both duty cycle and frequency influence performance.

The duty cycle regulates the output voltage, while the switching frequency affects efficiency, ripple, component size, and electromagnetic interference (EMI).

Key Differences

The following table summarizes the distinction between these two important PWM parameters.

Parameter	Controls	Typical Effect
Duty Cycle	Average power delivered	LED brightness, motor speed, heater power
Frequency	Number of switching cycles per second	Flicker, audible noise, waveform smoothness, switching losses

Understanding this distinction is fundamental to designing reliable PWM-based systems. In the next part of this chapter, we will examine additional PWM characteristics such as **resolution**, **phase**, and **polarity**, and learn how they influence the performance of real-world embedded applications.

Resolution

One of the most overlooked but extremely important PWM parameters is **resolution**. Resolution indicates **how many different duty cycle values** can be generated.

The higher the resolution,

- smoother brightness control,

- finer motor speed adjustment,
- better waveform quality.

For example,

8-bit PWM

An 8-bit timer can represent `2^8 = 256 levels`

Duty cycle values range from `0 to 255`

Each step changes the output by approximately

`100 / 255 ≈ 0.39%`

10-bit PWM

`1024 levels`

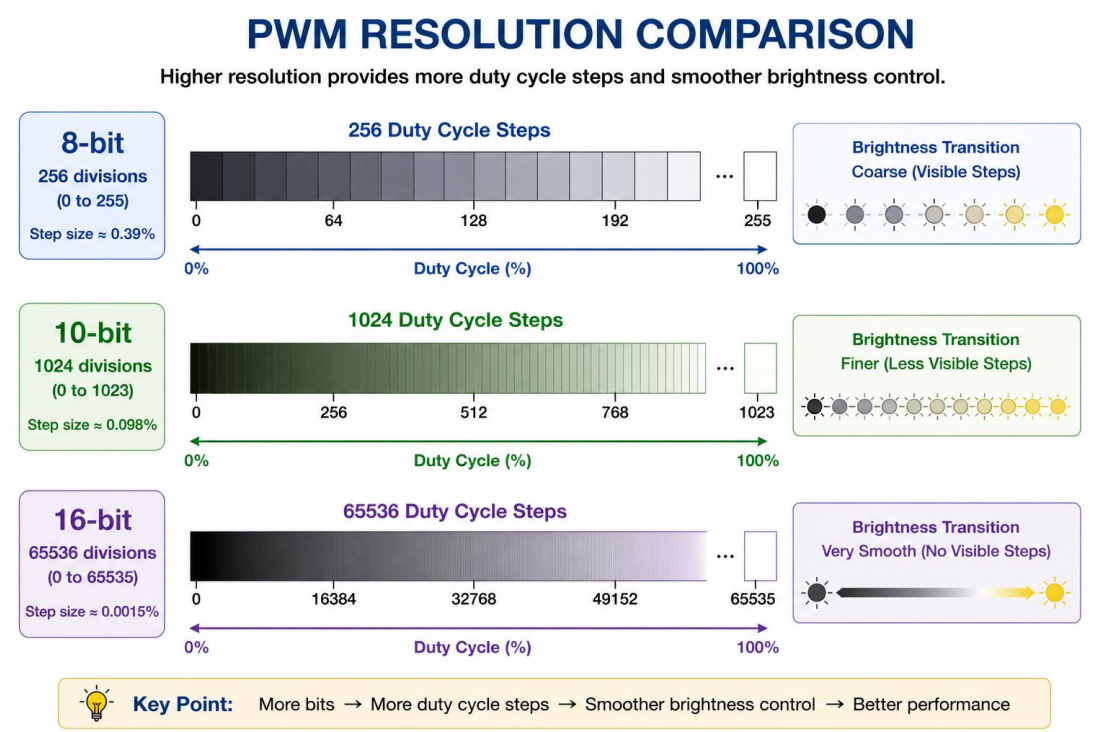
Each adjustment becomes much finer.

16-bit PWM

`65536 levels`

This extremely high resolution is ideal for

- precision motor control,
- waveform generation,
- laboratory equipment,
- industrial automation.



Polarity

PWM outputs may be configured as either

Active High

The output becomes HIGH during the ON interval.

Active Low

The output becomes LOW during the active interval. Different electronic circuits require different output polarities.

Fortunately, AVR timers allow this to be selected through configuration registers.

Phase

In systems containing multiple PWM outputs, the relative timing between different channels becomes important.

Two signals may

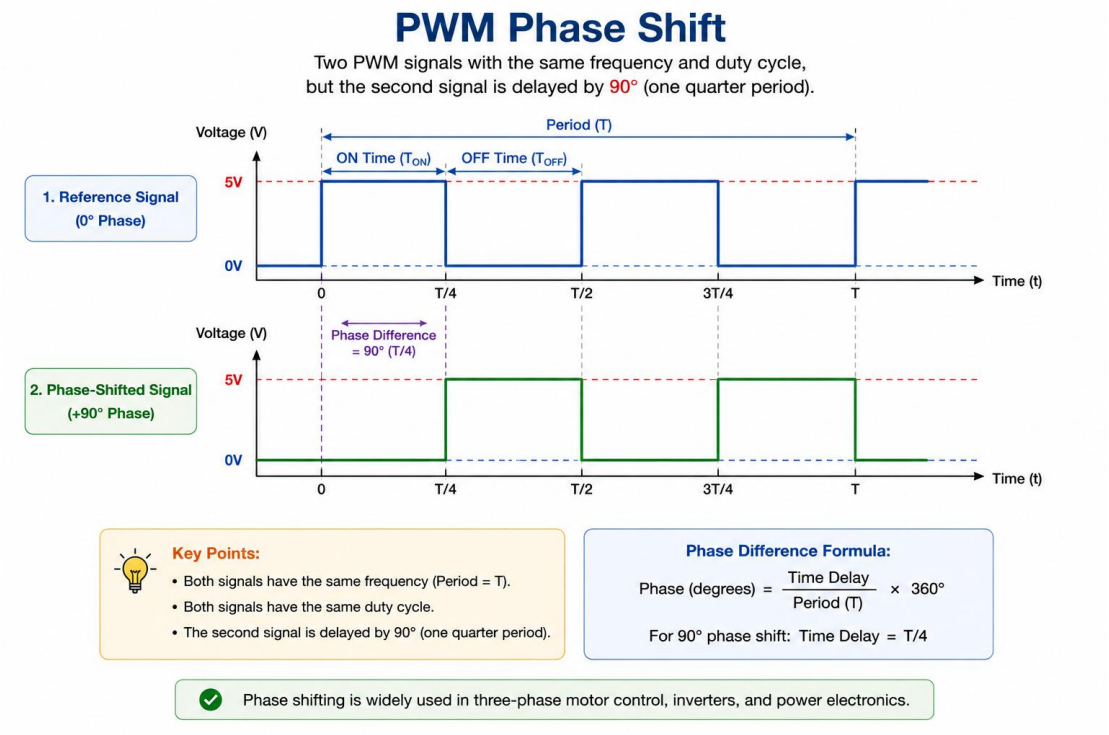
- start simultaneously,
- be shifted by 90°,
- be shifted by 180°,

- or have any other phase relationship.

Phase control is widely used in

- three-phase motor drives,
- inverters,
- switching power supplies,
- synchronous converters.

Although beginners rarely need to adjust phase, it becomes essential in advanced embedded and power electronics applications.



Summary of PWM Parameters

The following table summarizes the key PWM characteristics.

Parameter	Determines	Typical Applications
Frequency	Number of switching cycles per second	Flicker, noise, switching losses
Duty Cycle	Average power delivered	LED brightness, motor speed
Period	Time for one complete cycle	Timing calculations
Pulse	Duration of HIGH pulse	Servo motors, pulse control

Parameter	Determines	Typical Applications
Width		
Resolution	Number of duty cycle levels	Smooth control, waveform quality
Polarity	Active HIGH or LOW output	Circuit compatibility
Phase	Timing relationship between channels	Motor control, power electronics

Each parameter plays a unique role in the design of a PWM system. Selecting the correct combination ensures efficient operation, precise control, and optimal performance for the intended application.

PWM as an Energy Control Technique

It is tempting to think of PWM as a method for generating different voltages. In reality, PWM controls **energy transfer**.

Consider a water pump filling a bucket. Rather than reducing the water pressure, imagine opening and closing the valve rapidly. If the valve remains open only briefly during each cycle, less water enters the bucket.

If it remains open longer, more water flows. The water pressure never changes—only the duration for which the valve remains open.

PWM behaves in exactly the same way. The supply voltage remains constant. Only the duration of power delivery changes. This is why PWM is such an efficient method for controlling electrical power.

PWM Beyond Simple Control

Although PWM is widely used for dimming LEDs and controlling motors, its applications extend far beyond these examples.

By carefully varying the duty cycle over time, PWM can generate complex analog-like signals. When combined with filters and lookup tables, it becomes possible to produce:

- Sine waves
- Triangle waves
- Sawtooth waves
- Audio signals

- Arbitrary waveforms

These advanced applications transform a simple digital output into a versatile signal-generation tool. In later parts of this chapter, we will use the **MikroDuino PWM SDK** to generate practical waveforms, including filtered sine waves, demonstrating the full potential of hardware PWM in embedded systems.

Fast PWM vs Phase Correct PWM

Not All PWM Signals Are Generated the Same Way

So far, we have treated PWM as though every waveform is generated in exactly the same manner. In reality, AVR timers support several PWM operating modes. Among them, the two most commonly used are:

- Fast PWM
- Phase Correct PWM

Both generate PWM signals, but they differ in the way the timer counts.

This seemingly small difference has an important effect on frequency, waveform symmetry, and application suitability.

Understanding Timer Counting

Before comparing the two modes, let us briefly recall how a timer works.

A timer simply counts clock pulses.

When the count reaches a predetermined value, it either

- resets,
- changes direction,
- or generates an output event.

The counting sequence determines the resulting PWM waveform.

Fast PWM

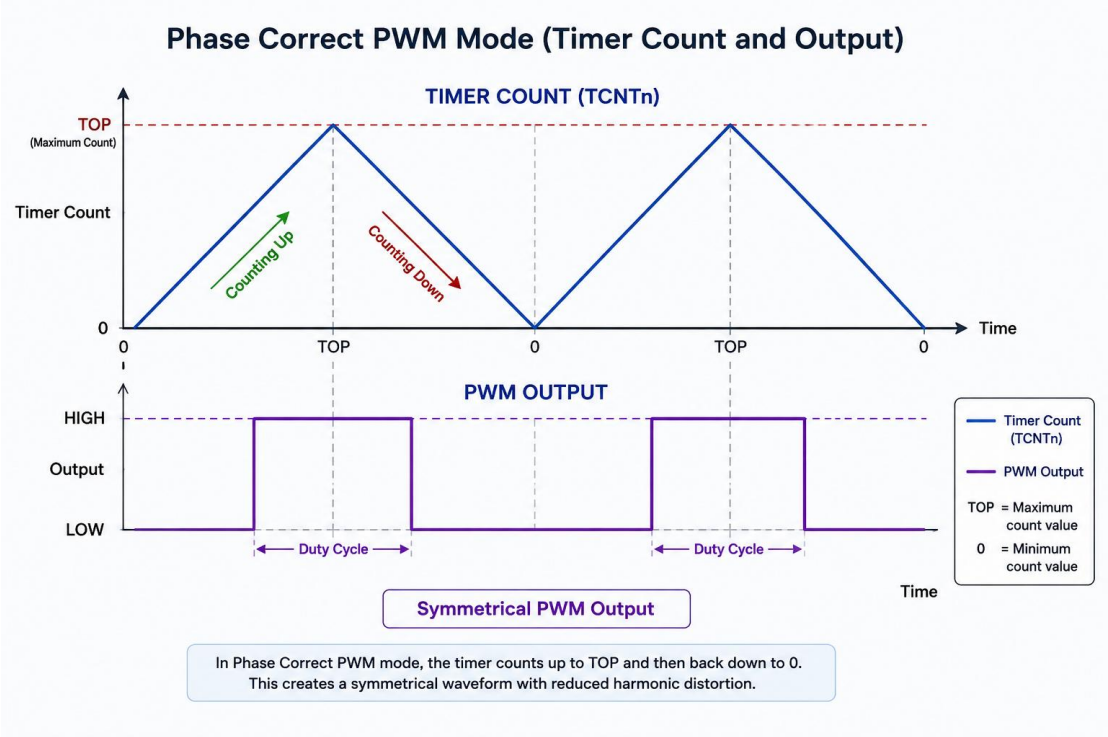
In **Fast PWM** mode, the timer counts only in one direction.

```
0

↓

1
```

```
↓
2
↓
...
↓
TOP
↓
0
↓
1
↓
2
↓
...
```

Once the counter reaches its maximum value (TOP), it immediately returns to zero and begins counting upward again.

This process repeats continuously.



Characteristics of Fast PWM

Because the timer counts in only one direction,

- PWM frequency is higher.
- Timer updates occur more frequently.
- Response is faster.
- Waveform is slightly asymmetric.

Fast PWM is ideal for applications where rapid response is more important than waveform symmetry.

Typical applications include:

- LED brightness control
- DC motor speed control
- Cooling fans
- General-purpose PWM outputs

Phase Correct PWM

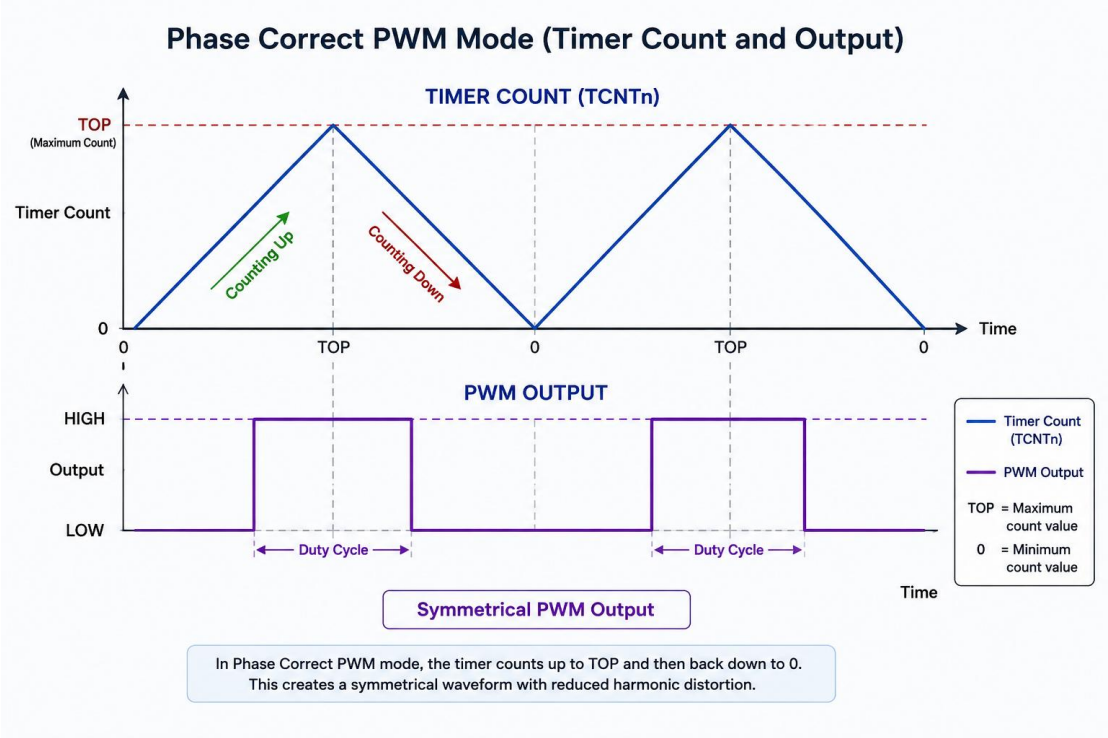
Phase Correct PWM operates differently. Instead of restarting immediately after reaching TOP, the timer reverses its counting direction.

```
0
↓
1
↓
2
↓
...
↓
TOP
↑
...
↑
2
↑
1
```

↑

0

The timer continuously counts up then down then up again. This produces a perfectly symmetrical PWM waveform.



Why is it Called "Phase Correct"?

The waveform generated by this method is symmetrical around the centre of every PWM period. The rising and falling edges occur at equal distances from the centre of the cycle. This balanced waveform significantly reduces timing errors and harmonic distortion.

Consequently, Phase Correct PWM is preferred whenever waveform symmetry is important.

Comparing the Two Modes

The following table summarizes the differences.

Feature	Fast PWM	Phase Correct PWM
Timer Counting	Up only	Up and Down
Output Frequency	Higher	Lower

Feature	Fast PWM	Phase Correct PWM
Waveform Symmetry	Slightly asymmetric	Perfectly symmetrical
Harmonic Content	Higher	Lower
Response Speed	Faster	Slightly slower
Best Applications	LEDs, motors, fans	Audio, waveform generation, power electronics

Choosing the Right PWM Mode

There is no universally "better" PWM mode. The choice depends entirely on the application.

Choose **Fast PWM** when:

- Higher PWM frequency is desired.
- Rapid response is important.
- LED dimming.
- Motor speed control.
- General-purpose PWM.

Choose **Phase Correct PWM** when:

- Symmetrical waveforms are required.
- Audio generation.
- Precision waveform synthesis.
- High-quality power electronics.
- Applications sensitive to harmonic distortion.

PWM Modes in AVR Microcontrollers

One of the strengths of AVR microcontrollers is their flexibility. By configuring a few timer control register bits, the same timer can operate in several different modes.

In later sections of this chapter, we shall learn how to configure these modes directly using AVR timer registers. After understanding the underlying hardware, we will see how the **MikroDuino PWM SDK** performs the same task using a simple, intuitive API, allowing complex PWM configurations to be created with only a few lines of code while still giving the programmer full control over the timer hardware.